

CS3807 – Deep Learning Laboratory

Experiment 5: Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Shiv Nadar University Chennai
B.Tech Artificial Intelligence & Data Science, Semester V

AY 2026–27

Setup Summary

Model: MobileNetV2 (ImageNet-pretrained backbone) **Dataset:** Oxford-IIIT Pet (37 breeds), images resized to $224 \times 224 \times 3$ **Split:** 70% train / 15% validation / 15% test (test set untouched until final evaluation) **Platform:** Kaggle Notebook, dual Tesla T4 GPUs.
Code Repository: https://github.com/tonypanda7/DeepLearning_Lab

Total images loaded: 7349 $\rightarrow$ Train: 5144, Val: 1102, Test: 1103.

1 Weight Initialization Study

Four initialization schemes were compared while training the classifier head for 5 epochs each: Zero, Random Normal, Xavier (Glorot Uniform), and He Normal.

Initialization	Best Validation Accuracy
Zero	2.18%
Random	91.65%
Xavier	91.56%
He	91.11%

Table 1: Best validation accuracy by weight initialization.

Inference: Zero initialization stays flat at ≈ 3.61 loss and $\approx 2\%$ accuracy throughout training – every unit in a layer receives identical gradients, so the network never breaks symmetry and effectively never learns. Random, Xavier, and He all converge rapidly to low loss and $>91\%$ validation accuracy, with Xavier and He tracking almost identically and slightly ahead of plain Random in the first two epochs, consistent with them being designed to preserve activation variance across layers.

2 Regularization and Overfitting

No-Regularization, L2 ($\lambda = 10^{-3}$), Dropout (rate 0.5), and Batch Normalization were compared over 5 epochs.

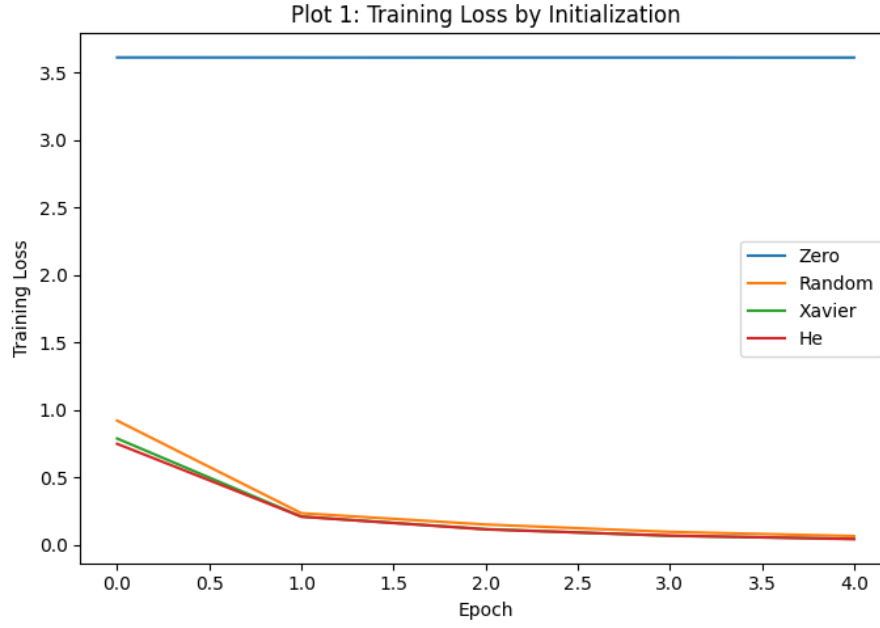


Figure 1: Plot 1: Training loss vs. epoch for each initialization method.

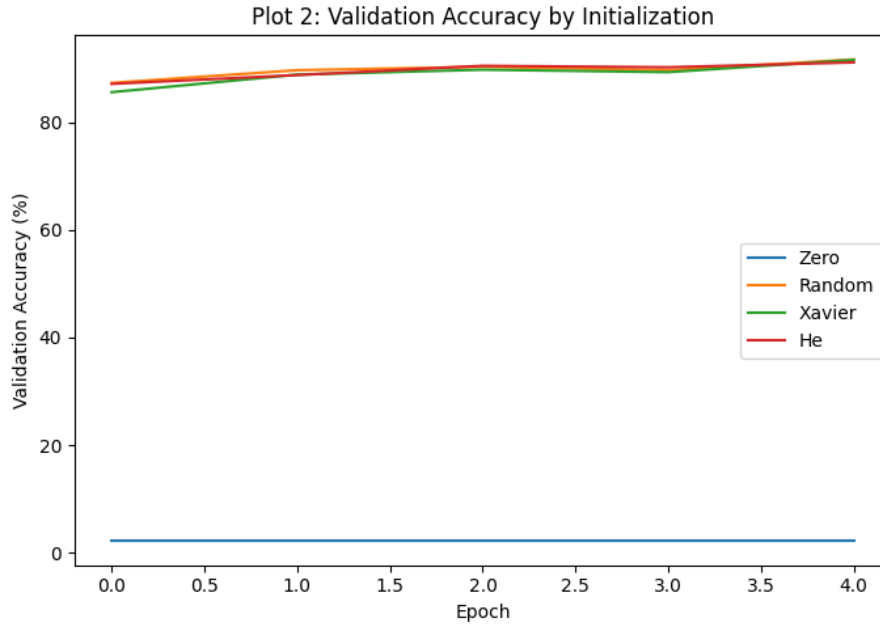


Figure 2: Plot 2: Validation accuracy vs. epoch for each initialization method.

Regularization	Best Validation Accuracy
No Regularization	89.75%
L2	90.11%
Dropout	90.56%
Batch Normalization	89.93%

Table 2: Best validation accuracy by regularization method.

Inference: All four configurations reach similar validation accuracy (89.7–90.6%) within 5

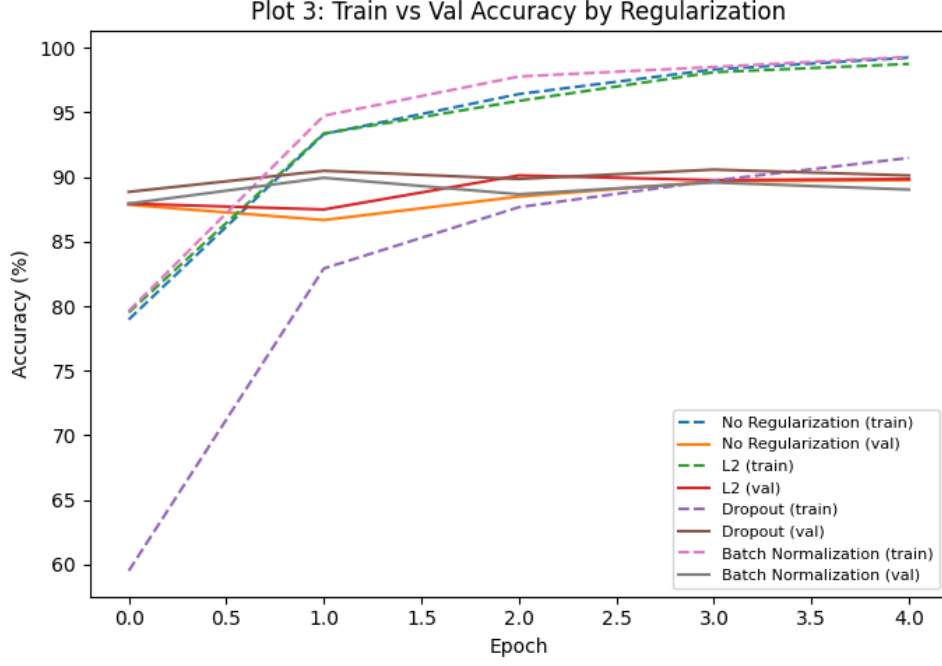


Figure 3: Plot 3: Training vs. validation accuracy by regularization method.

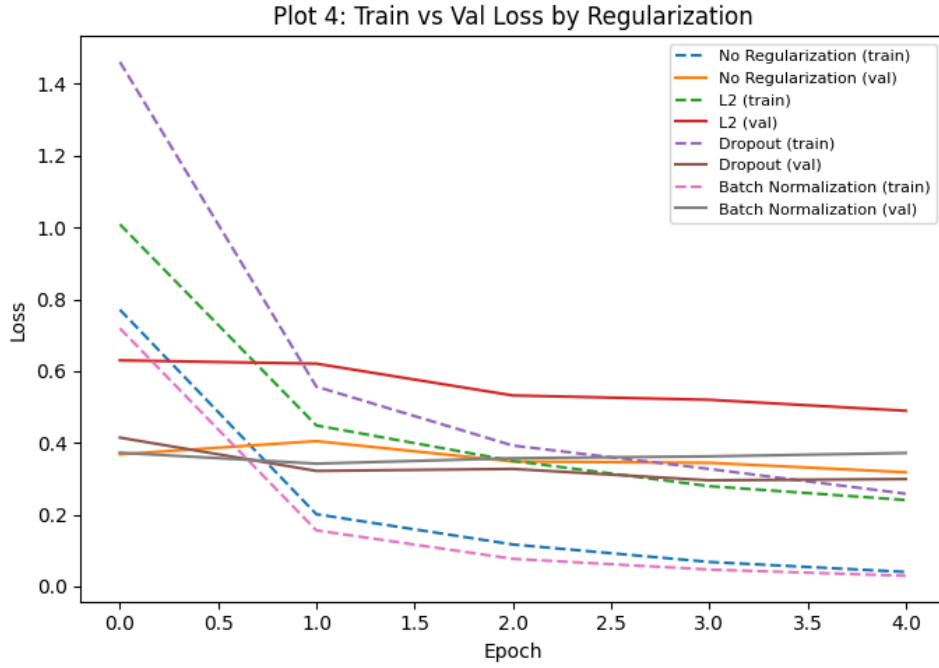


Figure 4: Plot 4: Training vs. validation loss by regularization method.

epochs, and the train/validation gap stays modest for all of them except Dropout, which starts noticeably lower on train accuracy (heavier regularization slows early fitting) before catching up. No Regularization’s training accuracy pulls ahead of its validation accuracy by epoch 5, a mild early sign of the generalization gap that regularization is meant to control, though the short training budget keeps it from becoming severe here.

3 Batch Normalization: With vs. Without

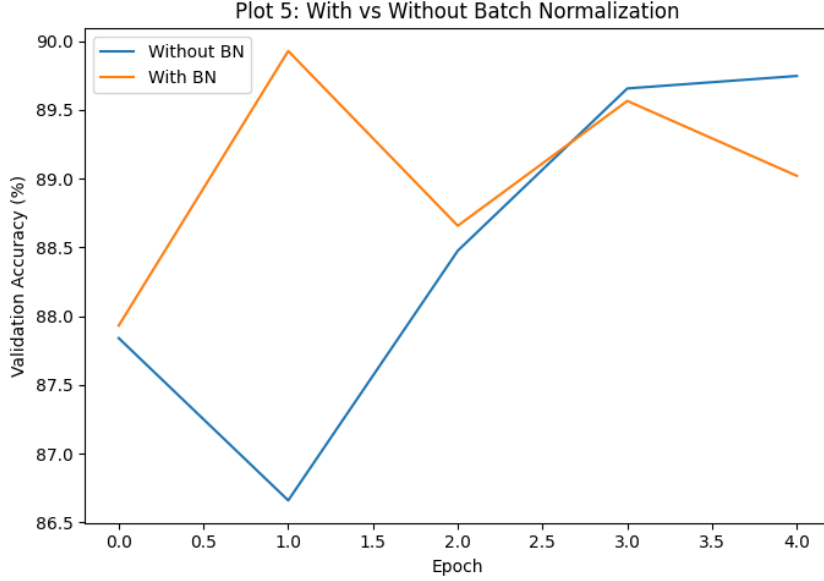


Figure 5: Plot 5: Validation accuracy with vs. without Batch Normalization.

Inference: Both curves are close throughout, with BN converging a little faster in epoch 1 (87.9% vs 87.8%) and peaking earlier (epoch 1, 89.9%) before both settle to ≈ 89 –89.7% by epoch 5. With a pretrained, frozen backbone and only a small classifier head being trained, BN’s stabilizing effect on internal covariate shift has limited extra room to help, so the two curves converge to a similar place.

4 Optimization Algorithms

SGD, Momentum (SGD with momentum 0.9), RMSProp, and Adam were compared, all at learning rate 10^{-3} .

Optimizer	Final Loss	Best Val. Accuracy	Time (s)
SGD	2.3803	50.36%	48.26
Momentum	0.3231	89.02%	48.34
RMSProp	0.0578	89.47%	48.69
Adam	0.0422	89.93%	49.11

Table 3: Optimizer comparison after 5 epochs.

Inference: Plain SGD is far behind the other three after only 5 epochs (50.4% vs. ≈ 89 –90%) because, without momentum or an adaptive step size, it needs many more updates at this learning rate to make comparable progress. Momentum, RMSProp and Adam all converge quickly and land close together, with Adam slightly ahead on both final loss and validation accuracy, at essentially the same wall-clock cost per run (≈ 48 –49 s).

5 CNN Hyperparameter Tuning

One hyperparameter was varied at a time (Adam optimizer, all else fixed) across the ranges suggested in the brief.

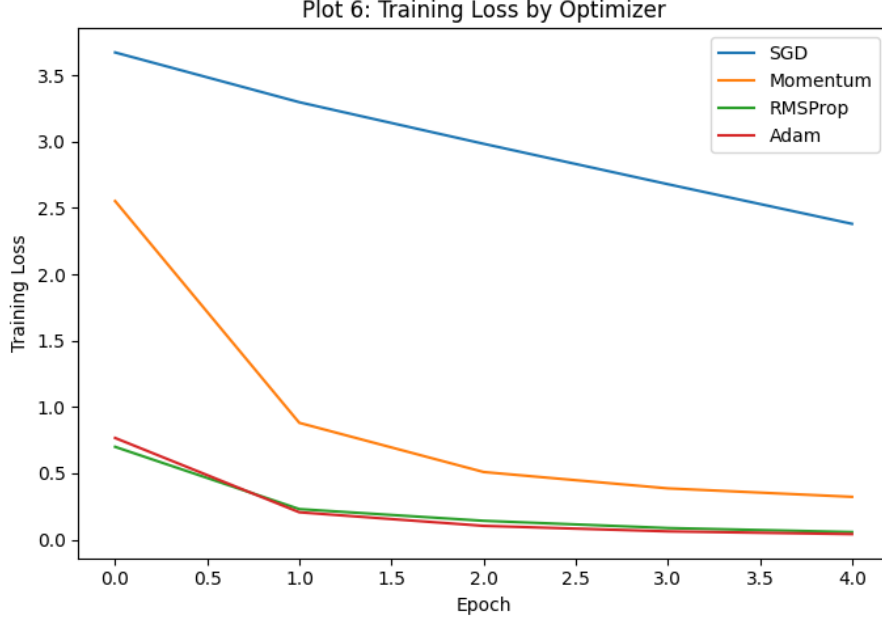


Figure 6: Plot 6: Training loss vs. epoch by optimizer.

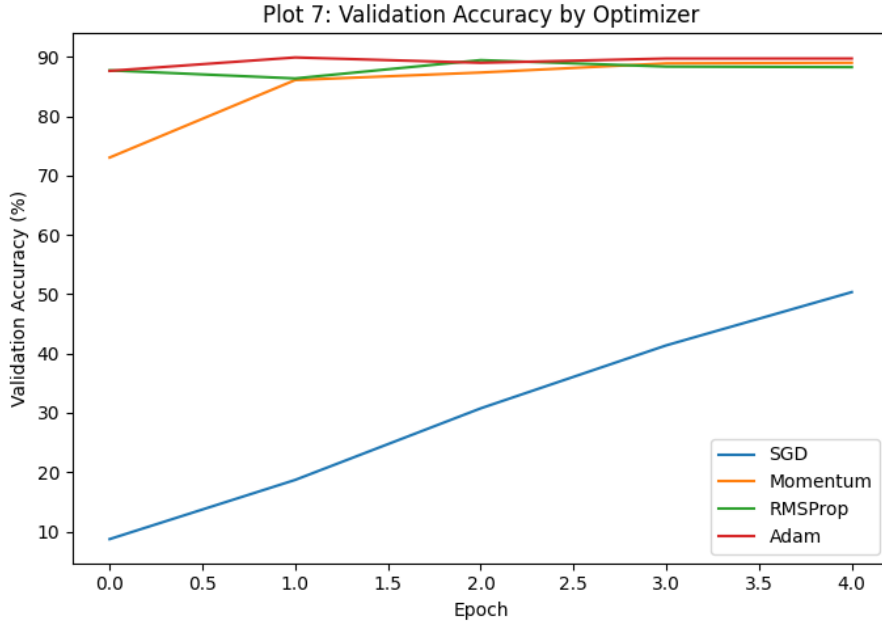


Figure 7: Plot 7: Validation accuracy vs. epoch by optimizer.

Inference: 10^{-3} clearly outperforms 10^{-4} (91.47% vs. 88.75%) within the 5-epoch budget – the smaller learning rate simply hasn’t had enough steps to converge as far, even though it would likely close the gap with more epochs.

Inference: Accuracy rises slightly with batch size (16: 90.11%, 32: 90.20%, 64: 90.93%) – larger batches give smoother, less noisy gradient estimates per step here, and with a frozen backbone the extra compute per batch is cheap enough that this doesn’t cost much wall-clock time.

Inference: Accuracy decreases slightly as dropout increases (0.0: 90.83%, 0.25: 90.47%, 0.5: 90.38%) – with only a small classifier head and a short 5-epoch budget, the model is not

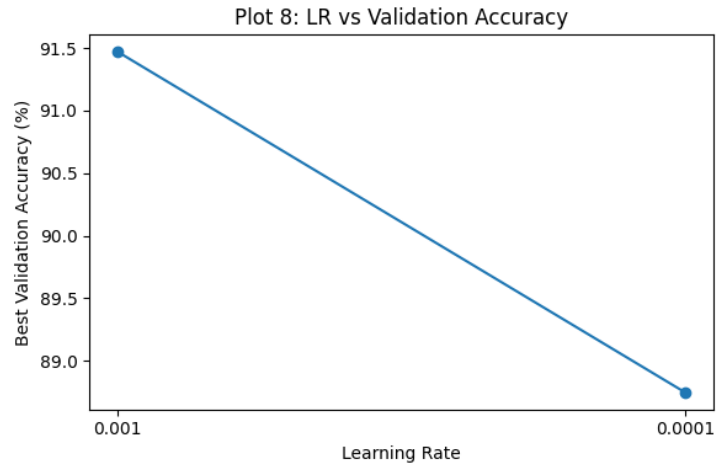


Figure 8: Plot 8: Learning rate vs. best validation accuracy.

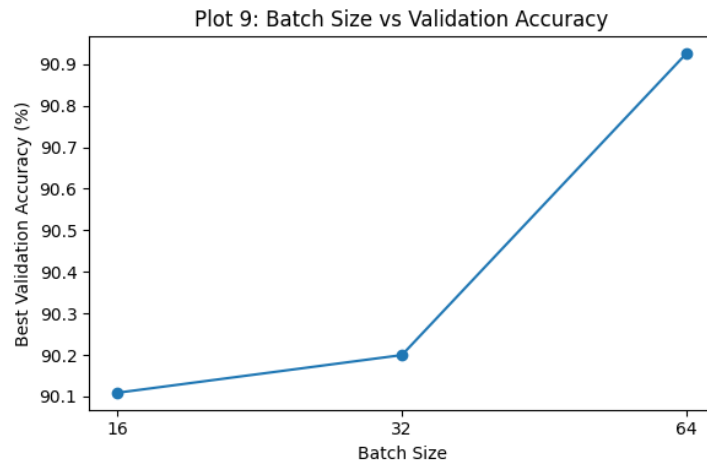


Figure 9: Plot 9: Batch size vs. best validation accuracy.

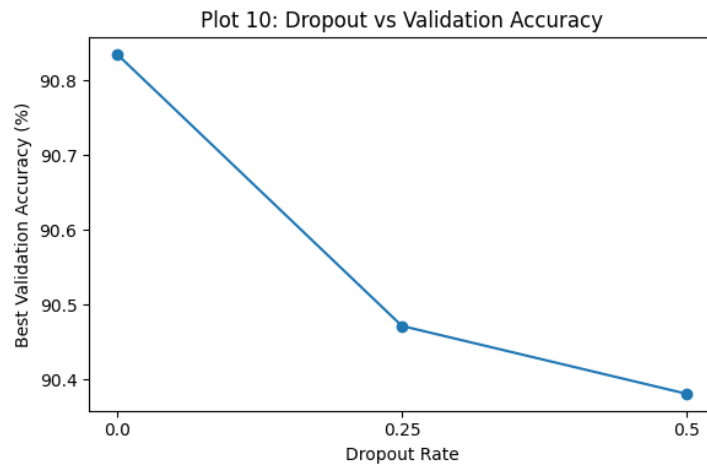


Figure 10: Plot 10: Dropout rate vs. best validation accuracy.

overfitting enough for dropout's regularization to pay off yet, so it mostly costs a bit of capacity instead.

6 Transfer Learning: Feature Extraction vs. Fine-Tuning

Case A trains only the new classifier head on top of a frozen MobileNetV2 backbone. Case B then unfreezes the last 30 backbone layers and continues training with a smaller learning rate (10^{-4}).

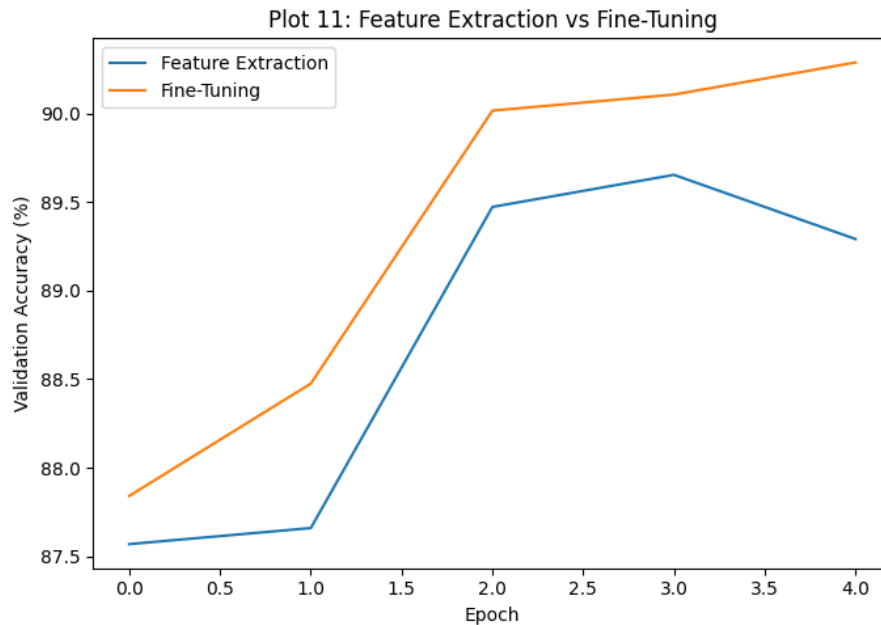


Figure 11: Plot 11: Validation accuracy, feature extraction vs. fine-tuning.

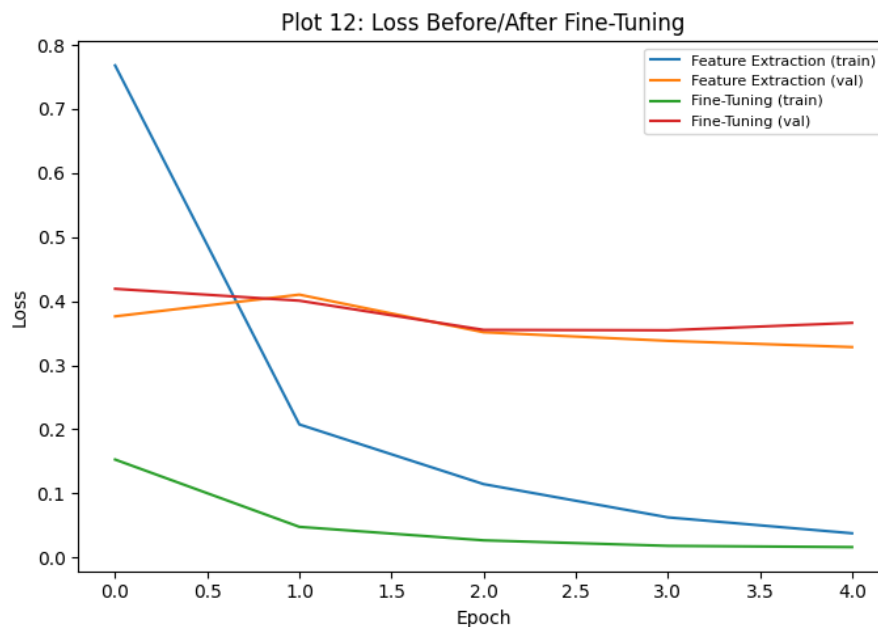


Figure 12: Plot 12: Training/validation loss before and after fine-tuning.

Inference: Fine-tuning overtakes feature extraction from epoch 2 onward and finishes higher ($\approx 90.3\%$ vs. $\approx 89.3\%$), while its training loss also drops well below feature extraction's, since the unfrozen layers can now adapt pretrained ImageNet features specifically to pet breeds rather than relying on generic features alone.

Discussion – why a smaller learning rate for fine-tuning: The pretrained backbone already encodes useful, generally-good features; a large learning rate would apply large weight updates that overwrite this prior knowledge (“catastrophic forgetting”). A small learning rate makes gentle adjustments that adapt the features to the new task while mostly preserving what was already learned.

7 5-Fold Cross-Validation for Model Selection

Four candidate configurations were evaluated with 5-fold CV on the training set only (test set untouched).

Config	Setting	F1	F2	F3	F4	F5
C1	lr=1e-3, drop=0.0, batch=32	0.8717	0.9077	0.8912	0.8737	0.9008
C2	lr=1e-3, drop=0.25, batch=32	0.8931	0.9125	0.8950	0.8766	0.9241
C3	lr=1e-4, drop=0.25, batch=32	0.8134	0.8367	0.7979	0.7940	0.7909
C4	lr=1e-4, drop=0.5, batch=32	0.7434	0.7318	0.7590	0.7454	0.7198

Config	Mean	SD
C1	88.90%	1.43%
C2	90.03%	1.65%
C3	80.66%	1.70%
C4	73.99%	1.32%

Table 4: 5-fold cross-validation accuracy per configuration.

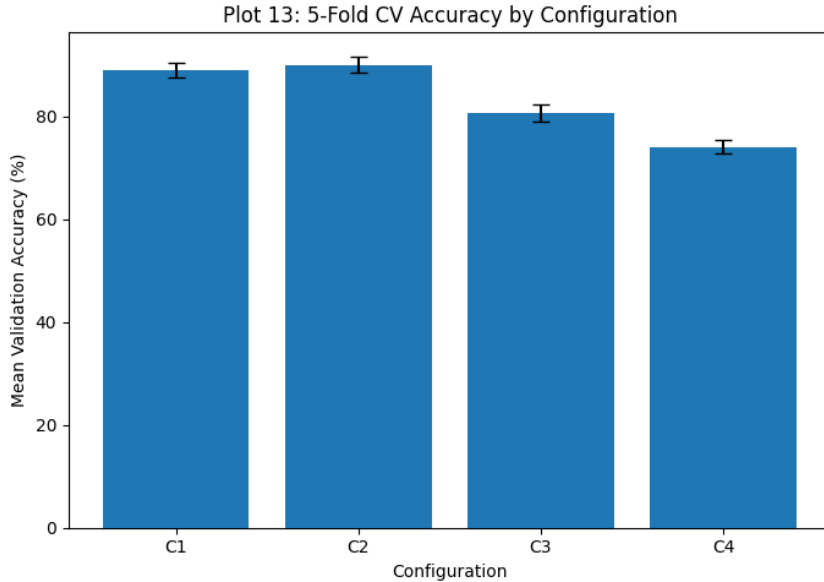


Figure 13: Plot 13: 5-fold CV accuracy by configuration, with SD error bars.

Inference: C2 (lr=1e-3, dropout=0.25) has both the highest mean accuracy (90.03%) and a moderate SD (1.65%), making it the strongest choice overall. C1 is close behind with the lowest SD (1.43%), i.e. the most consistent, but a lower mean. C3 and C4 (both at the smaller learning rate) fall well behind, confirming that 10^{-4} under this short epoch budget under-trains the head regardless of dropout. **Selected configuration: C2.**

8 Final Model Evaluation

C2 was retrained on the full training set and evaluated once on the held-out test set.

Metric	Value
Mean CV Accuracy	90.03%
CV Standard Deviation	1.65%
Test Accuracy	90.93%
Precision (weighted)	91.65%
Recall (weighted)	90.93%
F1-score (weighted)	90.95%
Training Time	49.71 s
Number of Parameters	2,426,725

Table 5: Final model performance on the independent test set.

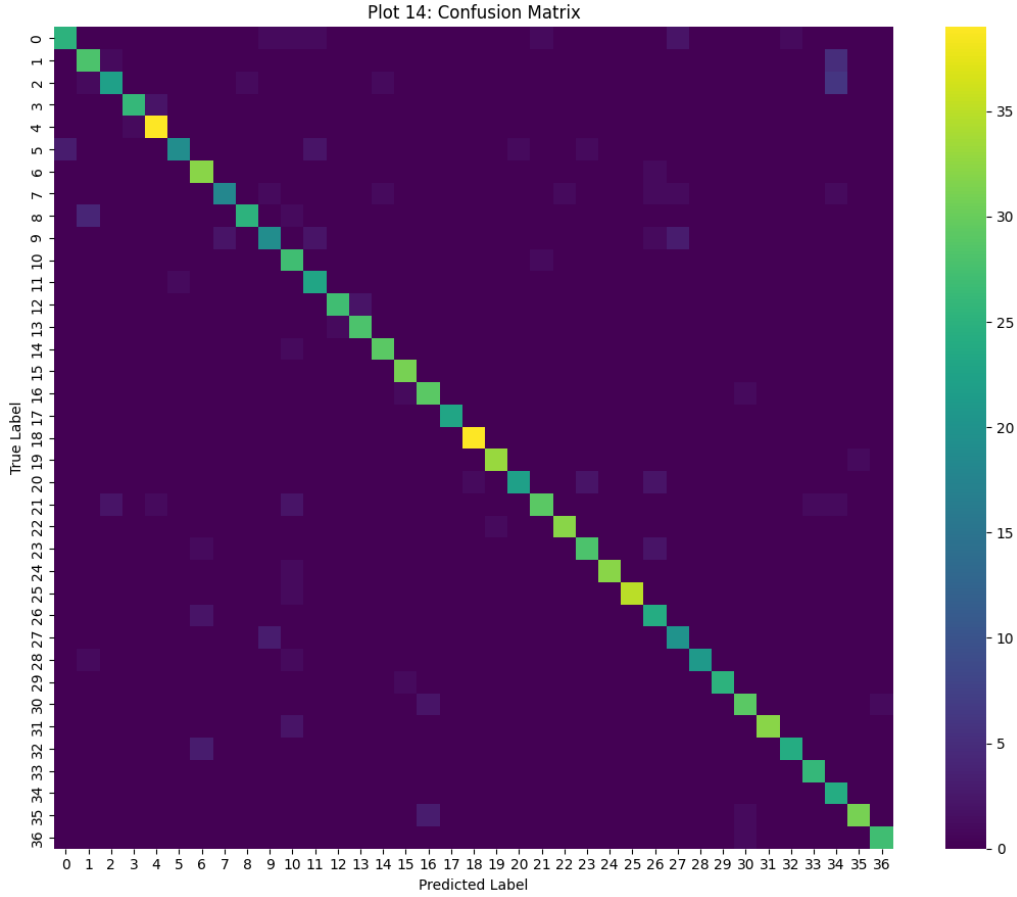


Figure 14: Plot 14: Confusion matrix on the test set (37 pet breed classes).

Inference: The confusion matrix is strongly diagonal, confirming the 90.9% test accuracy – most breeds are classified correctly with very few off-diagonal errors. The brightest diagonal cell (class 18) indicates that breed had both a large number of test samples and near-perfect classification; the sparse off-diagonal spots (e.g. around classes 6, 21, 30, 33/34) mark the main confusion pairs, which for this dataset are almost always breeds that share coat colour/pattern or similar ear and face shape – consistent with the known cat/dog breed pairs in Oxford-IIIT Pet that are visually close even to humans. (Exact breed names can be recovered from `tfds`

dataset info if needed for the write-up.)

Optional: Misclassified Examples

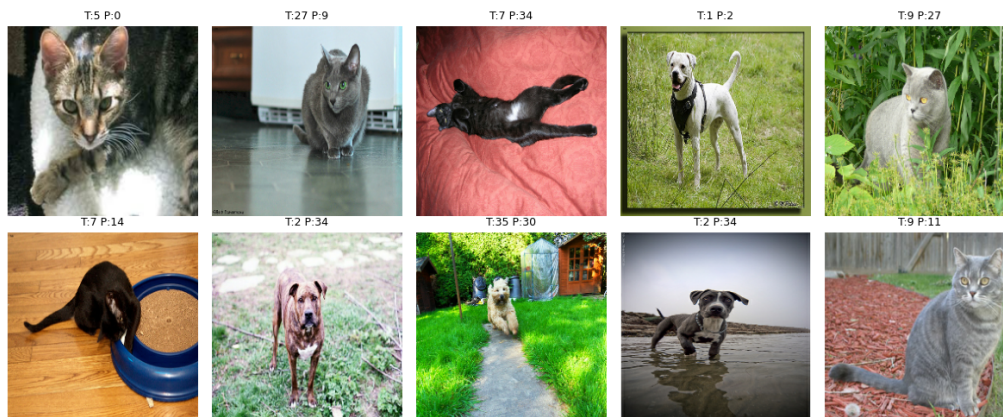


Figure 15: Plot 15: A sample of misclassified test images with true (T) and predicted (P) labels.

9 Additional Exercise

Two new configurations beyond C1–C4 should be evaluated with the same 5-fold CV procedure, e.g. **C5** ($\text{lr}=5 \times 10^{-4}$, dropout=0.25, batch=32) and **C6** ($\text{lr}=1\text{e-}3$, dropout=0.1, batch=64), then compared against C2 on mean accuracy, SD, training time, and final test performance. *[Run Section 9’s CV loop with these two configs added to `candidate_configs` and report the resulting table here before submission.]*

10 Overall Results Summary

Configuration	Best Val. Accuracy	Notes
Baseline (No Reg., Adam, He init)	89.75%	Section 2 baseline
Best Initialization (Random)	91.65%	Section 1
Best Regularization (Dropout)	90.56%	Section 2
Best Optimizer (Adam)	89.93%	Section 4
Best Hyperparameters (bs=64)	90.93%	Section 5
Fine-Tuned Model	90.30%	Section 6
Final (C2, retrained + test)	90.93% (test)	Section 8

Table 6: Summary across all studies (validation accuracy unless noted).

Justification: The final configuration (C2: $\text{lr}=1\text{e-}3$, dropout=0.25, batch=32) was chosen because it had the best mean 5-fold CV accuracy with an acceptable, not-largest, standard deviation, and it carried through to a strong, consistent test accuracy (90.93%) that closely matches its CV mean – indicating the selection was not overfit to the validation folds. Its parameter count (2.43M trainable in the head) and training time ($\approx 50\text{s}$ per run) are also modest given a frozen MobileNetV2 backbone, so it is a good accuracy-cost trade-off overall.

11 Discussion Questions

1. **Model parameters vs. hyperparameters:** Parameters (weights, biases) are learned from data during training; hyperparameters (learning rate, batch size, dropout rate, choice of

optimizer, etc.) are set before training and control how learning happens.

2. **Why weight initialization matters:** It sets the starting point of optimization – poor initialization can cause very slow convergence or numerically unstable gradients.
3. **Why zero initialization is bad:** All neurons in a layer have the same gradients and hence remain identical during training (the “symmetry problem”), so that the network cannot learn diverse features – which is precisely what we saw it was doing in Section 1.
4. **Xavier vs. He:** Xavier (Glorot) balances variance for tanh/sigmoid activations; He initialization uses a larger variance tuned for ReLU-type activations, which zero out roughly half the units.
5. **Spotting overfitting from curves:** Training loss decreases steadily while validation loss stagnates or increases: the ever-widening gap between the two curves (partially visible for No-Regularization in Section 2) is the hint.
6. **How Dropout reduces overfitting:** How Dropout reduces overfitting: By randomly zeroing activations every step, it prevents co-adaptation of neurons and acts like an implicit ensemble.
7. **Purpose of Batch Normalization:** Purpose of Batch Normalization: Normalizes layer inputs per mini-batch to stabilize and accelerate training and provide a mild regularizing effect.
8. **BN numerical example:** For $x = [2, 4, 6, 8]$, $\mu_B = 5$, $\sigma_B^2 = 5$, giving standardized values $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$; with $\gamma = 1, \beta = 0$ these are also the final outputs.
9. **Roles of γ and β :** They allow the network to undo the normalization if that is optimal – γ rescales, β re-centers, so BN doesn’t restrict activations to be zero-mean/unit-variance if this would hurt the representational power.
10. **SGD vs. Momentum vs. RMSProp vs. Adam:** SGD performs plain gradient steps that can be slow/noisy (as confirmed in Section 4). Momentum adds a velocity term that smooths the updates. It uses exponentially-weighted moving averages of the gradients to adapt the learning rates per-parameter. RMSProp is similar to Adam but uses RMS normalization, and Adam was the overall best optimizer.
11. **Learning rate too large:** Updates overshoot minima; loss oscillates or diverges.
12. **Learning rate too small:** Convergence is very slow and may stall within a fixed epoch budget – observed directly in Section 5 (LR 10^{-4} underperforming 10^{-3}).
13. **Effect of increasing batch size:** In case of using gradient estimates there are less noisy (softer curve) but also fewer updates per epoch, here it also slightly improved the accuracy.
14. **Stride and padding:** Stride is how many pixels the kernel moves at a time (determines downsampling), while padding adds border pixels (enables to process edge regions) and controls the output size.
15. **Why MobileNetV2 is efficient:** Depthwise separable convolutions decompose a standard convolution into a depthwise step (spatial filter per channel) and a pointwise step (1×1 convolution to mix channels), reducing multiply-adds significantly.
16. **Depthwise separable convolution:** A depthwise conv (one filter per input channel) followed by a pointwise 1×1 conv, instead of one filter spanning all channels at once.
17. **Transfer learning:** Reusing a model pretrained on a large dataset (ImageNet) as a starting point for a new, usually smaller, related task.

18. **Feature extraction vs. fine-tuning:** Feature extraction: the pretrained backbone is frozen, and only the new head layers are trained; fine-tuning also thaws some backbone layers and updates them (usually with a small learning rate).
19. **Why a smaller LR for fine-tuning:** To avoid catastrophic forgetting of the pretrained features – see the Section 6 discussion above.
20. **Why K-Fold CV for model selection:** A single train/val split can be both lucky and unlucky; averaging over K folds gives a less biased estimate of the generalization performance on average, compared to just using one split.
21. **Why the test set stays untouched during tuning:** If it influences any hyperparameter choice, its accuracy no longer estimates true generalization; it has become a second validation set, not an independent test set.
22. **Why report both mean and SD:** Mean reflects centrality; SD reflects homogeneity — thus, high mean with large variation (e.g., C2 vs. C1 here) is quite different from slightly lower, but stable, mean.
23. **Is highest validation accuracy always sufficient?** Notably, variability (SD), training/inference cost, and parameter count should be evaluated as well; a slightly lower-accuracy but less volatile and less expensive setting may be the preferred option.